1. Pre-Upgrade Checklist

Before upgrading your EKS cluster, ensure that the following steps are completed:

- **Backup Cluster State**: Backup etcd and other critical configurations if you're working with an on-prem Kubernetes setup.
- **Check Current Kubernetes Version**:
  To verify the current version of your cluster, use the following command:
- `eksctl get cluster --name <cluster-name> --region <region>`

  Or, via the **AWS Console** under **EKS > Clusters**.

- **Review Official Kubernetes and EKS Documentation**:
  Read the release notes for both Kubernetes and EKS to understand any breaking changes and the new features in the version you're upgrading to.
- **Document Running Add-ons**:
  List all the add-ons currently running in your cluster to ensure they are compatible with the new Kubernetes version:
- `kubectl get deployments -n kube-system`

  Key add-ons to check:

  - CoreDNS
  - VPC CNI (Amazon VPC CNI Plugin)
  - Metrics Server
  - Horizontal Pod Autoscaler (HPA)
  - Istio
  - KEDA
  - Karpenter
  - ISTIO

PATHNEX
Shaping Careers, Building Futures

---

2. Upgrade Add-ons First

Upgrade your add-ons before upgrading the control plane. Managed add-ons can be upgraded using `eksctl`, while other add-ons may require manual upgrades.

- **Upgrade Managed Add-ons Using `eksctl`**:

```
eksctl update addon --name coredns --cluster <cluster-name> --force --
region <region>
eksctl update addon --name kube-proxy --cluster <cluster-name> --force -
-region <region>
eksctl update addon --name vpc-cni --cluster <cluster-name> --force --
region <region>
```

- **Manually Upgrade Other Add-ons** (e.g., Istio, KEDA, Karpenter):
  For Istio, Prometheus, or KEDA, follow their respective upgrade documentation. For example:

```
kubectl apply -f https://github.com/aws/amazon-vpc-cni-
k8s/releases/download/v1.11.7/amazon-vpc-cni-k8s-1.11.7.yaml
```

---

3. Upgrade the EKS Control Plane

Once the add-ons are upgraded, proceed with upgrading the EKS control plane.

1. **Upgrade EKS Control Plane Using `eksctl`**:

```
eksctl upgrade cluster --name <cluster-name> --region <region>
```

   This command will upgrade the EKS control plane to the latest supported Kubernetes version. This is a **non-disruptive upgrade**, meaning that workloads will continue running while the upgrade happens in the background.

2. **Alternative: Upgrade via AWS Console**:
   - **Go to EKS > Clusters > Your Cluster**.
   - **Click "Update version"** and select the new Kubernetes version.
   - **Important**: EKS upgrades are sequential, meaning you cannot skip minor versions. For example, you must upgrade from v1.32 to v1.33 before upgrading to v1.34.

---

4. Upgrade Worker Nodes (Node Groups)

After upgrading the control plane, you will need to upgrade the worker nodes to ensure they are running the new Kubernetes version.

- **Managed Node Groups**:
  1. **Create a New Node Group**:
     Create a new node group using `eksctl` with the updated Kubernetes version:

```
eksctl create nodegroup \
  --cluster <cluster-name> \
  --name <new-nodegroup-name> \
  --node-type t3.medium \
  --nodes 3 \
  --nodes-min 1 \
  --nodes-max 5 \
  --node-ami-family AmazonLinux2 \
  --version <target-k8s-version> \
  --region <region>
```

  2. **Cordon and Drain Old Node Group**:
     Once the new node group is up and running, cordon the old nodes and drain them to safely evict running pods:

```
kubectl cordon <old-node-name>
```

PATHNEX
Shaping Careers, Building Futures

```
kubectl drain <old-node-name> --ignore-daemonsets --delete-
emptydir-data
```

3. **Delete the Old Node Group**:
   After the old nodes are drained, you can delete the old node group:

```
eksctl delete nodegroup --cluster <cluster-name> --name <old-
nodegroup-name>
```

- **Self-Managed Node Groups**:
  For self-managed node groups (EC2 instances in an Auto Scaling group):
  1. **Update the Launch Template** with the latest EKS-optimized AMI for the new Kubernetes version.
  2. **Update the Auto Scaling Group** to use the updated AMI.
  3. **Cordon and Drain Old Nodes** as with managed node groups.

---

5. Post-Upgrade Validation

After completing the control plane and node upgrades, validate that everything is working as expected:

1. **Check Node Status**:
   Make sure all nodes are in the "Ready" state:

```
kubectl get nodes
```

2. **Check Pod Status**:
   Ensure that all pods are running and there are no issues with your workloads:

```
kubectl get pods --all-namespaces
```

3. **Verify Add-ons**:
   Ensure that the managed add-ons (e.g., CoreDNS, VPC CNI) are running correctly:

```
kubectl get deployment -n kube-system
```

PATHNEX
Shaping Careers, Building Futures

Interviewer: Can you walk me through the process of upgrading a Kubernetes cluster on EKS?

ANSWER:

- **Pre-Upgrade**: First, I'd check the current Kubernetes version with `eksctl`, review the release notes, and document the add-ons in use (like **CoreDNS**, **VPC CNI**, **Metrics Server**, **Istio**, **KEDA, Karpenter, Autoscaller**).

- **Upgrade Add-ons**: I'd upgrade critical managed add-ons first using `eksctl` to ensure compatibility with the new version. For others like Istio or KEDA, I'd upgrade them manually.

- **Upgrade Control Plane**: Then, I'd upgrade the control plane using `eksctl` or the AWS Console. This step is non-disruptive, so workloads keep running.

- **Upgrade Worker Nodes**: After the control plane, I'd create a new node group with the updated version and migrate workloads by draining and deleting the old nodes.

- **Post-Upgrade Validation**: Finally, I'd validate the cluster by checking node and pod status, and ensuring add-ons are working properly.

- **Rollback**: If something goes wrong, I'd manually revert the Addons version or recreate node groups, since EKS doesn't support a direct rollback.

PATHNEX
Shaping Careers, Building Futures